

LESSON 6

Recursion & Binary Search

A function that calls itself. Every recursion needs a base case.

6.1 Recursion Mental Model

6.2 Factorial & Fibonacci

6.3 Binary Search

6.4 Power Function

6.5 Subsets & Permutations

6.6 Practice Problems

6.1 Recursion Mental Model

Every recursion has:

1. BASE CASE - when to STOP (prevents infinite loop)
2. RECURSIVE CASE - call itself with SMALLER input

Template:

```
def solve(problem):  
    if base_case(problem):      # STOP condition  
        return base_answer  
  
    smaller = break_down(problem) # Make it smaller  
    result = solve(smaller)      # Trust it works!  
    return combine(result)       # Build answer
```

Think of it like Russian nesting dolls:

- Open the big doll -> find a smaller doll
- Open that -> find an even smaller one
- ... until you reach the tiniest doll (BASE CASE)
- Then work backwards, closing each doll

6.2 Factorial & Fibonacci

```
# Factorial: n! = n * (n-1) * (n-2) * ... * 1  
def factorial(n):  
    if n <= 1:                # BASE CASE  
        return 1  
    return n * factorial(n - 1) # RECURSIVE CASE
```

```
factorial(5)  
5 * factorial(4)  
5 * (4 * factorial(3))  
5 * (4 * (3 * factorial(2)))  
5 * (4 * (3 * (2 * factorial(1))))  
5 * (4 * (3 * (2 * 1)))      <- base case!  
5 * (4 * (3 * 2))  
5 * (4 * 6)  
5 * 24  
120
```

```
# Fibonacci: 0, 1, 1, 2, 3, 5, 8, 13...
# SLOW recursive (exponential - shows understanding)
def fib(n):
    if n <= 0: return 0
    if n == 1: return 1
    return fib(n-1) + fib(n-2)

# FAST iterative (O(n) - use this in interviews)
def fib(n):
    if n <= 1: return n
    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b    # shift forward
    return b

# Memoized (O(n) with recursion - best of both worlds)
def fib(n, memo={}):
    if n in memo: return memo[n]
    if n <= 1: return n
    memo[n] = fib(n-1, memo) + fib(n-2, memo)
    return memo[n]
```

6.3 Binary Search (O(log n) - Must Know)

```
# Search sorted array by halving search space each step

def binary_search(nums, target):
    left, right = 0, len(nums) - 1

    while left <= right:
        mid = (left + right) // 2

        if nums[mid] == target:
            return mid        # found!
        elif nums[mid] < target:
            left = mid + 1     # target is in right half
        else:
            right = mid - 1    # target is in left half

    return -1 # not found
```

```
nums = [1, 3, 5, 7, 9, 11, 13], target = 9
```

```
left=0, right=6, mid=3: nums[3]=7 < 9 -> left=4
left=4, right=6, mid=5: nums[5]=11 > 9 -> right=4
left=4, right=4, mid=4: nums[4]=9 == 9 -> return 4!
```

Only 3 steps to search 7 elements!
 For 1000 elements: ~10 steps. For 1,000,000: ~20 steps.
 That's the power of $O(\log n)$.

6.4 Generate All Subsets (Backtracking)

```
# Input: [1, 2, 3]
# Output: [], [1], [2], [3], [1,2], [1,3], [2,3], [1,2,3]]

def subsets(nums):
    result = []

    def backtrack(start, current):
        result.append(current[:])    # add copy of current

        for i in range(start, len(nums)):
            current.append(nums[i])  # choose
            backtrack(i + 1, current) # explore
            current.pop()            # un-choose (backtrack)

    backtrack(0, [])
    return result
```